

AI ENGINEERING COHORT · DELIVERABLE 4

System Design

Conversational Memory Intelligence System

Abhijeet Hiwale

August 2026

design/system_design.pdf

1. Executive Summary and Scope

This system decides what conversational information is worth remembering, stores it safely, retrieves and ranks it against a live query, injects it into model context within a fixed budget, and manages how it changes or fades over time. It is the direct answer to Deliverable 1's core problem: a stateless assistant re-initializes its understanding of the user every session, forcing repeated re-explanation and producing contradictory suggestions once a decision is forgotten.

Deliverable 3's naive baseline confirmed four specific, measured failure modes that this design exists to fix — not hypothetical ones:

1. Stale facts outrank current facts when both are lexically similar to a query (33% failure rate on the baseline's contradiction cases).
2. Sensitive content has no mechanism to be caught before it's retrieved and injected (100% leak rate on the baseline's sensitive-info case).
3. The system has no concept of "nothing relevant exists" — it always returns its best available match regardless of actual relevance (50% false-positive rate on cold-start cases).
4. Cases that *did* pass did so because of lexical luck (distinctive vocabulary), not because the underlying mechanism is robust — this is a latent risk, not a solved problem.

System boundary

Outside this system: the end user, the calling LLM/assistant application, and the application's own conversation-turn handling. **Inside this system:** admission, storage, retrieval, ranking, context construction, and lifecycle management, all reachable only through the Memory Service interface described in Section 5. The calling application sends conversation turns in and a query out; it never talks to an internal component directly.

2. Users, Use Cases, and Non-Goals

Primary user: a single end user interacting with a conversational assistant across multiple sessions, where continuity of context (project state, preferences, prior decisions) has direct value — the scenario Deliverable 1's problem statement is built around.

In scope:

- Single-user conversational memory across sessions (the primary use case).
- Multi-tenant deployment (multiple users' memories coexisting, isolated from each other) — this is required by Deliverable 1's privacy constraint and Week 4's isolation research, even though the primary narrative use case is single-user.
- Text-based conversational memory only — no image, audio, or structured-document memory.

Explicitly out of scope for this design (deferred, not ignored — each has a stated reason):

- Multi-agent shared memory (one memory pool accessed by multiple cooperating agents). Week 4's research on cross-tenant/cross-agent memory poisoning applies here, but this design treats single-tenant, single-agent memory as the harder-to-get-right foundation first.
- Cross-modal memory (remembering things from images or audio). No research week covered this; adding it would require its own admission/representation research this project hasn't done.
- Real-time collaborative memory (multiple users editing a shared memory space simultaneously). Deliverable 1's scope never required this, and it introduces concurrency problems (conflicting simultaneous writes) this design doesn't attempt to solve.

3. Inputs, Outputs, Constraints, Priorities, and Success Metrics

Inputs: conversation turns (both user and assistant, per Week 1's finding that assistant-generated commitments need admission too), each turn's user ID and timestamp, and a per-query token budget from the calling application.

Outputs: a ranked, budget-respecting block of injected context for a given query; structured logs/traces of every memory operation (per Week 3's OpenTelemetry decision); and an explicit "nothing relevant" signal when no memory clears a configurable relevance threshold — a first-class output this design adds specifically because Deliverable 3 showed a naive system can't produce it.

Constraints (carried directly from Deliverable 1, unchanged): context/token budget, latency (retrieval sits on the critical path), cost/scale (bounded growth, not indefinite accumulation), privacy/isolation (per-user enforcement, not just correctness in the common case), correctness over time (staleness and relevance are ongoing judgment calls, not one-time decisions), and safety (memory can't be used to bypass safety behavior or resurface content the user didn't intend to persist).

Architectural priorities. These constraints conflict with each other in practice, so this design states explicitly what wins when they do:

PRIORITY	ATTRIBUTE
Highest	Privacy & tenant isolation
High	Correctness (contradiction/staleness handling)
High	Latency
Medium	Cost
Medium	Recall
Low	Storage efficiency

Concretely: this design will accept a recall loss to enforce a relevance floor or a privacy filter (Section 4's invariants), and will accept storage growth before it accepts a shortcut on correctness. This explains, for example, why Section 5's retrieval component applies a hard tenant pre-filter even where it costs a small amount of retrieval flexibility.

Success metrics, with target values. These are engineering objectives chosen to represent acceptable interactive performance, not values derived from an existing benchmark or a specific prior system — there's no published number this project is trying to match, so these targets should be read as this project's own bar, revisable once Deliverable 6 produces real measurements against them. They also need a caveat the numbers alone don't carry: Deliverable 3's naive baseline already hit 100% recall@5, but on a 10-case, hand-built dataset with no real scale or paraphrase pressure. The targets below are for a realistic-scale workload, not a bar the baseline already cleared.

METRIC	TARGET
Recall@5	> 90%
Precision@1	> 95%
Contradiction/staleness failure rate	< 5% (naive baseline measured 33%)
False-positive relevance rate	< 5% (naive baseline measured 50%)
Retrieval latency (p95)	< 100ms
Context construction latency	< 20ms
Sensitive-content leak rate	0% (naive baseline measured 100%)

These map directly back to Deliverable 1's actual user-facing goal — fewer re-explanations, no contradicted decisions — rather than being treated as ends in themselves, per Week 3's challenge notes on keeping proxy metrics honest.

4. Assumptions, Invariants, and Failure Modes

Assumptions (stated explicitly, since an unstated assumption is an unreviewed one):

- Dense embeddings are available from an external service or local model at acceptable latency (Deliverable 3's TF-IDF baseline was a sandbox-forced substitute, not the production plan).
- Clocks across the system are reasonably synchronized — `valid_until` supersession depends on consistent ordering of writes.
- Tenant IDs arriving at the Memory Service are already authenticated upstream; this design does not implement authentication itself, only enforces isolation given a trusted tenant ID.
- pgvector's performance holds at the dataset sizes this system expects at MVP scale (per Week 2's adopted decision) — this assumption has an explicit revisit trigger in Section 13 (capacity).

- Conversational turns are assumed to arrive in order, per user, at the admission boundary, for this MVP scope. Out-of-order or duplicate delivery is not handled by this design and is explicit future work, not a hidden limitation.

Invariants (things that must always hold, regardless of implementation detail):

- A memory belonging to tenant A must never be returned for a query from tenant B. (Week 4: namespace isolation is necessary but not sufficient — this invariant requires a deterministic pre-filter, not a similarity-based one.)
- A deletion request must remove a memory's influence from every derived artifact built from it, not just the source record. (Week 4's "backflow" finding — this is the hardest invariant to guarantee in the whole system, flagged honestly rather than assumed solved.)
- When two memories conflict, the system must be able to identify which is current without relying on an LLM's in-context judgment of freshness. (Week 3's benchmark finding: LLM-judged freshness fails even with explicit instructions.)
- The system must be able to say "nothing relevant" rather than always returning a best-available match. (Directly added in response to Deliverable 3's Failure 3.)
- Every retrieved memory must be traceable back to its originating conversation turn through its stored provenance record. (This turns Week 4's provenance-tagging decision into an enforced invariant rather than an optional field — directly closes the self-review finding that provenance "must change real downstream behavior, not just exist in the schema.")

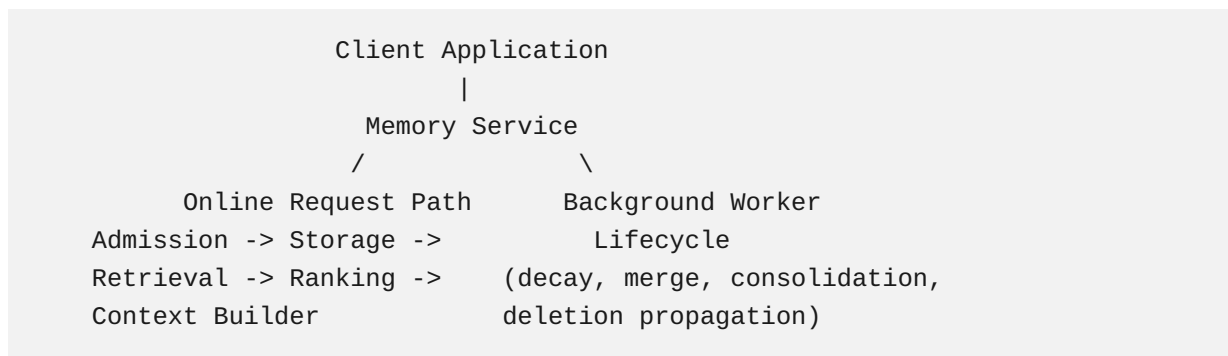
Failure modes, split by recoverability — a distinction the initial draft collapsed, and shouldn't have, since these are different engineering problems with different acceptable responses:

RECOVERABLE (DEGRADE GRACEFULLY, RETRY, OR CORRECT ON NEXT WRITE)	NON-RECOVERABLE (MUST BE PREVENTED STRUCTURALLY, NOT JUST HANDLED)
Retrieval miss (relevant memory not in top-k)	Cross-tenant leak
Relevance false positive (irrelevant memory surfaces)	Deletion propagation failure (backflow)
Stale memory outranks current, but current is still in top-k	Sensitive data persisted with no provenance trail to remove it
Budget truncation drops a lower-priority memory	

Recoverable failures degrade quality; non-recoverable failures are the ones the architectural priorities table above exists to protect against even at a cost to recall or latency.

5. Architecture Overview

The architecture consists of six loosely coupled components, each introduced to address a specific limitation identified during the research phase or Deliverable 3's evaluation, all orchestrated by a single **Memory Service** that owns the request flow — no calling application talks to an individual component directly.



Synchronous vs. asynchronous. Admission, storage writes, retrieval, ranking, and context construction all happen on the synchronous online path — a query cannot return until these complete, so their latency budget is real and enforced (Section 3's targets). Decay, merge, consolidation, and deletion propagation run as an asynchronous background process, per Week 1's original lifecycle design — they are not on any request's critical path, which is precisely what keeps the online path's latency bounded regardless of how much lifecycle work is pending.

1. **Admission** — classifies each candidate memory as ADD / UPDATE / DELETE / NOOP (Week 1), requires entity/slot-linking as a prerequisite capability, tags every record with provenance (Week 4), and applies PII detection as a pre-guardrail before anything is stored (Week 4). *Input: a conversation turn. Output: a memory operation (one of ADD/UPDATE/DELETE/NOOP) plus its provenance tag.*
2. **Storage** — pgvector-backed (Week 2, MVP-scoped), partitioned by tenant with deterministic filtering (Week 2 + Week 4 amendment), storing typed records with provenance and validity windows attached. *Input: a memory operation. Output: a stored memory ID and confirmation.*
3. **Retrieval** — hybrid BM25 + dense + RRF fusion (Week 2), no LLM inference on the critical request path, which keeps retrieval latency low and deterministic. Applies an explicit, configurable relevance threshold below which nothing is returned (new — closes Deliverable 3's Failure 3 gap). *Input: a query plus tenant ID. Output: a ranked candidate set, or an explicit empty result.*
4. **Ranking & conflict resolution** — deterministic `valid_until` supersession (Week 1) with a low-confidence fallback that keeps both records retrievable rather than silently closing one out (per Week 1's self-review). *Input: retrieval's candidate set. Output: a final ordered list with superseded records marked or excluded.*
5. **Context construction** — explicit per-zone token budgets including a dedicated tool-output zone (Week 2), applying the context-rot finding as a hard design constraint rather than relying on window size. *Input: the ranked list plus the caller's token budget. Output: the injected context block.*
6. **Lifecycle & consolidation** — four-lever framework (importance/merge/decay/eviction, Week 1), decay as reduced accessibility distinct from hard deletion (Week 1), unverified/low-provenance memory expiring by default (Week 4), and deletion propagation through consolidation lineage (Week 4's hardest open requirement). *Input: storage state on a schedule, plus explicit deletion requests. Output: updated/merged/evicted records, propagated through any derived artifacts.*

Each component's detailed responsibilities, interfaces, and data flow are specified further in `architecture.pdf` and `data_flow.pdf`. Section 6 below summarizes the responsibility boundaries.

6. Component Responsibility Boundaries

A short table clarifying what each component is and is not responsible for, since several of Deliverable 3's failures came from responsibilities that were implicitly nobody's job:

COMPONENT	RESPONSIBLE FOR	EXPLICITLY NOT RESPONSIBLE FOR
Admission	Classifying ADD/UPDATE/DELETE/NOOP, provenance tagging, PII pre-filtering	Ranking retrieved results, deciding what's "relevant" to a query
Storage	Persistence, tenant partitioning, validity-window fields	Similarity scoring, relevance judgments
Retrieval	Hybrid search, relevance floor, tenant pre-filter	Deciding which of two conflicting results is current (that's Ranking)
Ranking & conflict resolution	Supersession, low-confidence fallback, final ordering	Storage, admission-time filtering
Context construction	Token budgeting, zone allocation, truncation	Which memories were retrieved in the first place
Lifecycle & consolidation	Decay, merge, eviction, deletion propagation	Real-time retrieval-path decisions (this runs as a background process, not inline — see Section 5's sync/async split)

Design-process notes — which decisions came from external review vs. original drafting, and why — are logged in `design/decision_records/` rather than inline here, per this project's own accountability requirement (Section 12 of the handbook) to record material AI assistance and the decisions that followed from it.

7. Data Model, Provenance, and Lifecycle

Core record — every stored memory carries these fields:

FIELD	PURPOSE	TRACES TO
<code>id</code>	Unique identifier	—
<code>tenant_id</code>	Hard partition key for isolation	Week 4, Section 4's cross-tenant invariant
<code>text</code>	The memory content itself	—
<code>dense_embedding</code> , <code>sparse_terms</code>	Two representations for hybrid retrieval	Week 2's BM25+dense+RRF decision
<code>provenance</code>	One of: <code>user_stated</code> , <code>assistant_generated</code> , <code>tool_derived</code> , <code>retrieved_document</code>	Week 4; now an enforced invariant (Part 1, Section 4)
<code>admission_op</code>	The operation that created/last touched this record: ADD, UPDATE, DELETE, NOOP	Week 1
<code>valid_from</code> , <code>valid_until</code>	Explicit validity window; <code>valid_until</code> is null until superseded	Week 1's deterministic supersession decision
<code>confidence</code>	Admission-time confidence score, used for the low-confidence supersession fallback	Week 1 self-review
<code>pii_scan_result</code>	Pass/flag/redacted, plus which detector version ran	Week 4's PII detection decision
<code>status</code>	<code>active</code> , <code>decayed</code> , <code>merged</code> , <code>evicted</code> , <code>deleted</code>	Week 1's four-lever consolidation; <code>merged</code> is added here explicitly — it was implicit in the <code>data_flow</code> 's background-flow behaviour (source records are retained as <code>merged</code> rather than deleted when a consolidated record is created) but not named in the original table
<code>consolidation_lineage</code>	List of source record IDs this record was derived from, if any	Week 4's deletion-propagation ("backflow") requirement

Why `consolidation_lineage` exists and what it actually has to do: this is the field that makes deletion propagation possible at all. When the Lifecycle component merges or summarizes several raw memories into one consolidated record (Week 1's "merge" lever), the consolidated record's lineage field points back to every source it was built from. A deletion request against any source record triggers a lookup through lineage: every derived record that included the deleted source gets re-consolidated or evicted, not just the source itself. This is still the least proven part of the whole design — Week 4 flagged this as

adopted-as-a-requirement with the actual mechanism only prototyped, and that status hasn't changed here. This data model makes the requirement *implementable*; it doesn't yet make it *guaranteed correct* — that's Deliverable 6's job to verify.

Lifecycle, tied to the four-lever framework (Week 1):

- **Importance** — decided at admission, informs whether a memory is a decay candidate at all.
- **Merge** — background process combines related raw memories into a consolidated record, writing `consolidation_lineage`.
- **Decay** — reduces a memory's rank/accessibility over time (soft); does not delete it. Now also triggered by low provenance/unverified status expiring by default (Week 4).
- **Eviction** — the actual removal path, either from decay reaching a floor or from an explicit deletion request; the only lever that touches `consolidation_lineage` propagation.

8. Data Flows

Part 1's diagram simplified the online path as one linear chain. That's accurate for a single query, but it conflates two distinct triggers that don't happen at the same time or on the same event:

Write flow (triggered by a new conversation turn):

```
Conversation turn -> Admission (classify + provenance tag + PII scan) -> Storage
```

Read flow (triggered by a query needing memory context):

```
Query -> Retrieval (hybrid search + tenant pre-filter + relevance floor)
      -> Ranking (supersession check + multi-signal scoring)
      -> Context Builder (per-zone token budgeting)
      -> injected context returned to caller
```

Background flow (scheduled, not triggered by any single request):

```
Lifecycle Worker: scans Storage on a schedule
  -> applies decay / merge / eviction
  -> on any deletion, walks consolidation_lineage and propagates
```

The Lifecycle Worker is specified here as a scheduled/polling process against Storage, deliberately not committed to a specific message-broker technology (Kafka, RabbitMQ, or otherwise) — that's an implementation decision for Deliverable 5/6, not an architectural one, and none of the four research weeks evaluated message-queue infrastructure specifically. What this design does commit to: the worker runs asynchronously, off the request path (Section 9's synchronous/asynchronous split), and its schedule interval is a tuning parameter against Section 13's cost/latency trade-off, not a fixed design constant.

A single conversational exchange typically triggers both a write (the user's new turn gets admitted) and a read (the assistant needs context to respond) — but they are independent operations against the same Storage component, not steps in one pipeline. This matters operationally: a slow admission shouldn't block a query's retrieval latency budget, and the two should be allowed to run concurrently rather than serialized.

9. Retrieval, Ranking, and Context-Budget Policy

Retrieval (Week 2's adopted mechanism, applied in full): 1. Tenant pre-filter applied first and deterministically — not folded into similarity scoring (Week 4's amendment to Week 2's namespace isolation; this order matters, since filtering after ranking would mean a cross-tenant candidate was scored at all, which is the exact risk Week 4 flagged). 2. BM25 and dense similarity run in parallel over the tenant-filtered set. 3. Reciprocal Rank Fusion combines the two rankings. 4. A configurable relevance floor is applied to the fused result — anything below it is dropped, producing the explicit "nothing relevant" output when everything falls below threshold (closes Deliverable 3's Failure 3). No LLM call anywhere in this path (Week 2).

Ranking & conflict resolution (Week 1's adopted mechanism): 1. For each retrieved candidate, check `valid_until`. If a candidate has been superseded (another record with the same entity/slot has a later `valid_from`), it's excluded from the top position — unless supersession confidence was low at write time, in which case both records remain retrievable and are marked as conflicting rather than one being silently hidden (Week 1 self-review fallback). 2. Final ordering combines fused retrieval rank, recency, and a provenance-weighted trust factor (user-stated ranks above tool-derived, all else equal) — this is the multi-signal ranking Deliverable 1 required and the naive baseline never had. Concretely, each candidate's final score is:

```
final_score = w_r * rrf_rank_score + w_t * recency_weight(valid_from) + w_p *
provenance_trust(provenance_type)
```

where `rrf_rank_score` is retrieval's fused rank (Section 9, Retrieval), `recency_weight` decays as `valid_from` ages (mirroring Week 1's decay lever rather than introducing a second decay mechanism), and `provenance_trust` is a fixed lookup by provenance type (Section 7). The weights `w_r`, `w_t`, `w_p` are not fixed here — Deliverable 1's architectural priorities (Part 1, Section 3) rank correctness above recall, so the starting point is `w_r` and `w_t` weighted above `w_p`, tuned against Deliverable 6's evaluation set rather than guessed in advance. This formula is a structural commitment (three named, traceable signals, not an opaque score); the exact weights are an empirical question this design defers to where empirical answers belong.

Context construction (Week 2's adopted mechanism): Per-zone token budgets, each with a hard ceiling: system prompt, retrieved memory, conversation history, tool-output (its own zone, per Week 2 — not folded into history), current input, output reserve. Retrieved memories are injected in final-ranked order until their zone's budget is exhausted; nothing overflows into another zone's allocation. The context-rot finding (Week 2) is why this system never treats a larger context window as a substitute for this budgeting discipline, regardless of the underlying model's window size.

10. API Contracts (Summary)

Full request/response schemas are specified in `api_contracts.md`. At a high level, three operations, matching the two online flows plus explicit deletion:

- **POST /memory/turns** — admits a conversation turn. Returns the admission operation applied (ADD/UPDATE/DELETE/NOOP) and the record ID.
- **POST /memory/query** — the read flow. Takes a query and a token budget, returns either an injected context block or an explicit `no_relevant_memory` result (not an error — a valid, expected outcome per Deliverable 3's Failure 3 and Part 1's invariants).
- **DELETE /memory/{id}** — triggers eviction and lineage-based propagation. Returns once propagation through the lineage graph completes, or an explicit status indicating propagation is still in progress for large lineage chains, rather than a false-positive immediate success.

Error semantics principle: "nothing relevant found" and "tenant isolation prevented a match" are not represented as errors anywhere in this API — they're valid, expected outcomes with their own response shape. Only genuine failures (storage unavailable, malformed input) are errors. This distinction exists specifically because Deliverable 3 showed what happens when a system has no way to express "nothing relevant" as anything other than returning its best (wrong) guess.

11. Privacy, Authorization, Isolation, and Threat Model (Summary)

Full threat model is specified in `threat_model.md`. Summary of what this design commits to, directly from Week 4's research:

- **Tenant isolation** is a deterministic pre-filter (Section 9), not a similarity-adjacent guarantee — this is a non-recoverable-class protection (Part 1, Section 4).
 - **PII handling** applies regex + NER at admission (pre-guardrail) and again at retrieval/injection (post-guardrail), with an explicit minimum precision/recall bar validated against this project's own conversational data before shipping — not trusted on a published benchmark number alone (Week 4's PIIBench finding).
 - **Provenance** is used, not just stored: it gates ranking trust weighting (Section 9) and decay/expiry eligibility (Section 7), closing the "unused field" risk flagged in Week 4's self-review.
 - **Deletion propagation** through `consolidation_lineage` is this design's least-proven guarantee, named honestly as such rather than assumed solved.
 - Authorization itself (verifying a tenant ID is legitimate) is explicitly out of this system's scope per Part 1's assumptions — this design enforces isolation given a trusted tenant ID, it does not authenticate that ID.
-

12. Observability and Offline Evaluation

Tracing (Week 3's adopted mechanism): every memory operation — admission, ranking decision, supersession, decay, deletion — is a typed OpenTelemetry span nested under the parent request. Memory content lives in span *events*, not span *attributes*, with redaction/hashing applied at the collector level. This is more than a logging choice: Section 11's threat model treats collector misconfiguration as a real risk, not a footnote, per Week 4's self-review of Week 3's own decision. Tail-based sampling (capture all errors, sample routine traffic) applies once real traffic volume exists; not a concern at MVP scale.

Open item carried forward honestly: Week 3's self-review flagged that OTEL's "low cost" claim assumed a framework choice this design still hasn't made. That's still true here — auto- instrumentation cost is unknown until an implementation framework is picked in Deliverable 5/6. This section commits to OTEL as the standard; it does not yet commit to an instrumentation cost estimate, because that would be a number invented without a basis.

Offline evaluation uses the metric set established across Week 3 and Deliverable 3: recall@5, precision@1, the contradiction/staleness rate, latency p50/p95, token efficiency, and the minimal task-level check (does the system act on the current fact, not a stale one). Per Week 3's revised decision, full task-level evaluation — memory *use*, not just recall, tested against genuine multi-step scenarios — is deliberately deferred to Deliverable 6, where a real implementation exists to test against.

13. Capacity, Latency, Cost, and Operational Recovery

Storage capacity: pgvector at MVP scale (Week 2), with an explicit re-validation trigger — Week 2's self-review flagged that the cited recommendations assumed generic RAG workloads, not this system's comparatively write/update/delete-heavy access pattern (driven by supersession and lifecycle operations). This design does not treat "pgvector is fine" as settled; it's the correct starting point, re-checked once Deliverable 6 produces real update/delete volume.

Latency budget: matches Part 1's targets (retrieval p95 < 100ms, context construction < 20ms). The write flow (admission) has no equivalently strict budget, since it's decoupled from the read flow's request/response cycle (Section 8) — a slow admission delays when a memory becomes retrievable, not the latency of an unrelated concurrent query.

Cost: bounded primarily through the four-lever lifecycle (decay and eviction prevent unbounded storage growth) rather than through aggressive upfront filtering — this follows directly from Part 1's priority table, which ranks storage efficiency lowest. Namespace-based tenant isolation stays cheap at moderate scale; the documented trigger for upgrading to physical isolation (Week 4) is tenant count or compliance requirements crossing a threshold to be set once real usage data exists.

Operational recovery — a distinction this design has to get right, not just describe: a retrieval-path *error* (storage unreachable, index corrupted) and a *valid* "no relevant memory found" result must never be indistinguishable to the caller. Section 10 already draws this line in the API contract; it matters again here because a naive failure-recovery strategy (silently return empty results on any retrieval error) would quietly reintroduce Deliverable 3's Failure 3 through the back door — a caller couldn't tell "genuinely nothing relevant" from "the system is broken and failing closed." Errors must surface as errors.

Deletion consistency window: Deliverable 6's acceptance check requires deletion to remove a memory "within the documented consistency window." This design commits to stating that window explicitly once Deliverable 6 measures real lineage-propagation time — it is not fixed here, since inventing a number without Deliverable 6's data would be exactly the kind of unearned precision this project has avoided elsewhere (see Deliverable 1's treatment of context-budget zone sizes).

14. Alternatives Considered

Full decision records are in `design/decision_records/` (ADR-per-decision). Summary of the major alternatives this design rejected, and why — each traceable to a specific week's research rather than asserted:

ALTERNATIVE	REJECTED IN FAVOR OF	WHY
LLM-judged freshness/recency at retrieval time	Deterministic <code>valid_until</code> supersession	Week 3's benchmark finding: LLM freshness judgment fails even with explicit instructions
Full conversation history replay	Selective retrieval with budgeting	Deliverable 1's historical chain — replay breaks the token budget as conversations grow
Keyword-only (lexical) search	Hybrid BM25 + dense + RRF	Deliverable 1 required semantic matching; pure keyword search misses paraphrased queries
Dedicated vector database at MVP launch	pgvector on existing Postgres	Week 2 — near-unanimous recommendation at this system's expected scale; avoids premature infrastructure
Cross-encoder/ColBERT reranking at launch	Deferred, revisit if Deliverable 6 shows a measured gap	Week 2 — real technique, but "add a reranker" was flagged as a reflexive answer, not a measured need
Similarity-only tenant isolation	Deterministic pre-filter, applied before ranking	Week 4 — similarity search is probabilistic; multi-tenancy needs a hard boundary
Trusting a single PII benchmark number	Mandatory validation on this project's own conversational data	Week 4's PII Bench finding — detector accuracy is unstable out-of-distribution

15. Milestones, Gates, and Acceptance Tests

Proposed build sequence (to be structured into Genesis loops in Deliverable 5, not committed to a calendar here):

1. **M1 — Storage + Admission.** Typed records, provenance tagging, PII pre-guardrail, tenant partitioning. Gate: a stored record round-trips with all required fields intact.
2. **M2 — Retrieval + Ranking (core memory loop).** Hybrid search, relevance floor, supersession logic. Gate: Deliverable 3's exact failing cases (stale-outranks-current, false-positive relevance) now pass against this implementation.
3. **M3 — Context construction.** Per-zone budgeting, tool-output zone. Gate: budget ceilings are never exceeded under the long-conversation stress case from Deliverable 3.

4. **M4 — Lifecycle + deletion propagation.** Four-lever consolidation, lineage tracking. Gate: a deletion request against a source record removably affects every derived artifact — this is the milestone most likely to slip, given Week 4's honest assessment of its difficulty.
5. **M5 — Privacy hardening.** PII detector validated against real conversational data, threat model's non-recoverable failure modes specifically tested.
6. **M6 — Observability.** OTEL instrumentation, redaction at the collector.
7. **M7 — Full verification.** Deliverable 6's acceptance checks run in full, benchmarked directly against Deliverable 3's naive baseline numbers.

Acceptance tests are Deliverable 6's own required checks, restated here as the target this design is built to satisfy: relevant memories outrank distractors; stale preferences never override current ones; no cross-tenant memory returned under adversarial queries; deletion removes a memory from both source and retrieval paths within the (to-be-measured) consistency window; context selection respects the token budget; sensitive-data policy is tested with positive and negative cases; results are compared against Deliverable 3's baseline, not evaluated in isolation.

16. Risks and Unresolved Questions

Named honestly rather than smoothed over:

- **Deletion propagation correctness** (Week 4's hardest problem) is this design's single biggest risk. `consolidation_lineage` (Section 7) makes it implementable; nothing in this document makes it proven. M4 is where this either holds up or doesn't.
- **Entity/slot-linking for admission** — Week 1's self-review flagged this as a load-bearing prerequisite for both the ADD/UPDATE/DELETE/NOOP classification and supersession detection, and it still has no detailed design of its own. This needs its own decision record before M1/M2 implementation starts in earnest.
- **Dense embedding availability** — Part 1's first assumption. Deliverable 3's baseline used TF-IDF only because this sandbox had no network path to an embedding service; the production design assumes one exists, and specifies the fallback rather than leaving it implicit: if the embedding call fails or times out, Retrieval (Section 9) falls back to BM25-only for that request — degraded to lexical-only per-request, not a system-wide outage — and the failure is surfaced through the same error path as any other retrieval error (Section 10's error-semantics principle: this is a real failure, not silently reinterpreted as "nothing relevant"). This keeps the system available under a degraded mode rather than failing the request outright, at the cost of reopening Deliverable 1's original "keyword search misses paraphrases" failure mode for the duration of the outage — an explicit, bounded trade-off rather than an unstated one.
- **PII detector validation** is specified as a requirement (Section 11) but has no validated detector yet — this can't be resolved on paper, only once Deliverable 6 has real conversational data to test against.
- **Pre-filtered search efficiency** — Week 4's self-review flagged that deterministic tenant filtering needs checking against whatever index implementation gets chosen; pgvector's specific behavior under tenant pre-filtering hasn't been verified, only assumed reasonable.

- **Target metric values** (Part 1, Section 3) are engineering objectives, not derived from a benchmark — they may need revision once Deliverable 6 produces the first real measurements against a realistic-scale workload rather than Deliverable 3's ten hand-built cases.